

Group Name: Player

Project 1 Part 3 Documentation

Name of the player program: Player (Player.java)

Additional files/program: State.java, GameConstants.java

I. Members of the Group

Since the changes were minimal, the efforts of the team in this new submission just consist of heuristic weight improvements. Meaning everyone contributes a little bit evenly in testing and debugging.

II. Instructions for Compilation

Use any java compiler with the Player.java file:

1. Run the file Player.java in the compiler (i.e IntelliJ)
2. The program will start and the user can interact with the program through the terminal. The user can type “orange” or “blue” to tell the program which stone it should be.
3. The user can interact with the program to play Lasker Morris (by entering the specified move format)

To run the program with referee:

1. Enter “**cs4341-referee laskermorris -p1 "java Player.java" -p2 "java Player.java" --visual**” in terminal to run the referee.
2. The referee will call two of the Player program to play against each other.
3. Once the game ends, the referee will return the result of the game.

III. CHANGES: Utility and Evaluation Improvements

We updated the weights of each heuristics for the program to go for win/losses instead of draw.

We have updated the Number of Blocked Pieces heuristic to the number of pieces that are NOT blocked.

1. Stone(s) Left in Hand:
 - Closed Mill x 50
 - Number of Mills x 40
 - Number of Non-Blocked Pieces x 3
 - Number of Pieces Left x 10
 - Two-piece Configuration x 10
 - Three-piece Configuration x 7
 - Number of Double Mills x 0
2. No Stones Left in Hand:

- Closed Mill x 55
- Number of Mills x 70
- Number of Non-Blocked Pieces x 10
- Number of Pieces Left x 15
- Two-piece Configuration x 0
- Three-piece Configuration x 0
- Number of Double Mills x 8

3. Flying:

- Closed Mill x 40
- Number of Mills x 0
- Number of Non-Blocked Pieces x 0
- Number of Pieces Left x 0
- Two-piece Configuration x 10 (to avoid opponent's capturing)
- Three-piece Configuration x 1 (to avoid opponent's capturing)
- Number of Double Mills x 0

All of them have Win Condition x 5000

The heuristic utility is the sum of all the heuristic criterias with their multipliers within the stage that the state belongs to.

The heuristic evaluations criterias and multipliers are inspired by this research paper: <http://www.dasconference.ro/papers/2008/B7.pdf>. We modified the factors to fit with a Lasker Morris Game instead of Nine Men's Morris.

IV. Results

TESTINGS

The program was tested against human players, itself through the referee, Gemini, and the old Minimax algorithm.

1. Playing against humans.
 - We did various different moves, while: playing seriously or losing on purpose to see if the program tries to win (i.e leaving mills purposefully open).
 - The program usually makes optimized moves, which beats the human player.
2. Playing against itself through the referee.
 - The program usually draws against itself.
 - This is expected because our heuristic was meant to fix the late game where it should go for win instead of draw.
3. Playing against the old Minimax algorithm

- The program usually draws against the old minimax algorithm
 - Since the program draw at exactly 20 moves, the referee closed it too soon for it to proceed
 - The two program keeps blocking each other
 - This is expected because our heuristic was meant to fix the late game where it should go for win instead of draw.
4. Playing against Gemini
- We detected that the program keeps trying to draw instead of win here.
 - Against the random move of Gemini, it did not try to win as much, so we changed the weights for it to win more.
 - This new program should win against Gemini most of the time.

V. Important notes:

- Due to slow initialization of the program to the referee in the beginning, we set the time limit of the program to ~2.3 seconds, and increased the time limit to 4.9 seconds after the first move was returned. The program is able to handle the full 5 seconds outside this context.
- The function to check legality of moves: `checkLegalMoves()` is fully functional however due to bugs with the referee's initialization as mentioned above does not work with it. So for the code running with the referee the ability to `checkLegalMoves()` has been commented out.